

Knowledge Distillation from a Pretrained ResNet34 to a Lightweight ResNet18 for Image Classification

Weidi Yan
Auckland, New Zealand
wadeyan03@gmail.com

Abstract—This paper presents a knowledge distillation approach for image classification by transferring learned representations from a pretrained ResNet34 teacher model to a lightweight ResNet18 student model. Using a hybrid loss combining cross-entropy and Kullback-Leibler divergence, the student network is trained to mimic the teacher’s soft predictions. Experimental results demonstrate that the distilled student model achieves competitive accuracy with significantly fewer parameters and improved computational efficiency. The method proves effective for building compact models suitable for resource-constrained environments without compromising performance.

I. INTRODUCTION

Image recognition has undergone significant development in the field of computer vision and has found widespread applications in areas such as medical diagnostics, autonomous driving, and industrial automation. With the emergence of deep learning techniques, convolutional neural networks (CNNs) have achieved remarkable success in image classification and recognition tasks. However, as the demand for higher accuracy increases, state-of-the-art CNN models often require deeper architectures with more convolutional layers to reach marginal improvements in performance. This trend leads to substantial increases in model size, computational cost, and memory usage, making such models impractical for deployment on resource-constrained devices.

To address these challenges, knowledge distillation has emerged as an effective model compression technique. It enables the transfer of learned knowledge from a large, well-performing “teacher” model to a smaller, more efficient “student” model, without significant loss in accuracy. This approach retains much of the predictive power of the teacher while greatly reducing the computational overhead.

This project focuses on the implementation of a knowledge distillation framework for image recognition tasks, using a pretrained ResNet34 as the teacher model and ResNet18 as the student model. The distillation process transfers the predictive behavior of the teacher to the student by training the latter to approximate the soft output distributions produced by the former, alongside supervision from ground truth labels.

II. RELATED WORK

Knowledge distillation (KD) is a model compression technique that allows a small student network to learn from a large teacher network, typically by mimicking softened outputs

rather than hard labels. The foundational work by Hinton et al. [1] introduced the use of temperature-scaled softmax to generate “soft targets” that reveal inter-class similarities. In their formulation, the student minimizes a weighted sum of two losses: the standard cross-entropy with true labels and the Kullback–Leibler divergence between the teacher and student soft distributions.

Ba and Caruana [2] showed that even shallow networks can approximate deep ensembles by learning to regress on the logits of the teacher. Romero et al. [3] proposed FitNets, which extend KD by transferring intermediate representations (“hints”) from the teacher’s hidden layers to guide early layers in the student. Zagoruyko and Komodakis [4] explored attention-based distillation, while Furlanello et al. [5] introduced Born-Again Neural Networks via iterative self-distillation. Deep Mutual Learning [6] further explores joint training of two students to learn collaboratively.

Despite their success, many of these methods focus on large-scale tasks with complex loss functions and architectures. Most existing distillation methods are designed with complex models, large-scale datasets, or specialized loss functions such as feature- or attention-based transfers [3], [4]. In contrast, this project revisits the classical logit-based distillation proposed by Hinton et al. [1], applying it directly to a small-scale image classification task with only 10 classes and fixed 224×224 resolution. Despite its simplicity, we show that this basic approach, which does not use feature matching or additional regularization, can significantly improve student model performance when transferring knowledge from a pre-trained ResNet34 teacher to a compact ResNet18 student.

III. METHODOLOGY

In this work, a ResNet34 model pretrained on the ImageNet dataset is used as the teacher network. Knowledge distillation is conducted by transferring the soft outputs (soft labels) of the teacher to a lightweight student model, ResNet18. This approach aims to improve the classification performance of the student by guiding it to approximate the output behavior of the teacher. During training, both hard labels and soft targets are utilized to enhance the learning process and boost generalization capability.

A. Data preprocessing

The dataset preprocessing pipeline follows a comprehensive seven-step procedure to convert raw image data into a format suitable for deep learning model training and evaluation. The dataset contains 10 object classification categories: *african elephant*, *airliner*, *banana*, *convertible car*, *golden retriever*, *goldfish*, *parachute*, *rugby*, *sunglasses*, and *tiger cat*.

Each category includes 780 training images and 260 validation images, resulting in a total of 7,800 training samples and 2,600 validation samples. Additionally, 2,600 test images are provided for final evaluation.

The preprocessing steps are as follows:

- **Step 1: Image Loading.** A custom `image_loader` function is used to load images and labels from hierarchical folder structures. It uses OpenCV's `cv2.imread()` to read images in BGR format and includes logic to handle different directory structures depending on the presence of class subdirectories.
- **Step 2: Image Resizing and Cropping.** The `img_resize` function first scales the shortest edge to 224 pixels, then applies center cropping to obtain a 224×224 image. The crop coordinates are computed as:

$$x = \frac{\text{new_width} - 224}{2}, \quad y = \frac{\text{new_height} - 224}{2} \quad (1)$$

where `new_width` and `new_height` are the resized image dimensions.

- **Step 3: Normalization.** Images are converted from unsigned 8-bit integers (`uint8`) to 32-bit floats (`float32`), and pixel values are scaled to the $[0, 1]$ range by dividing by 255.0. This ensures numerical stability and accelerates convergence.
- **Step 4: Tensor Format Conversion.** Image arrays are converted from NumPy's NHWC format to PyTorch's preferred NCHW format using `tensor.permute(0, 3, 1, 2)`. This rearranges the channel dimension for efficient GPU access.
- **Step 5: Per-Channel Standardization.** Channel-wise mean and standard deviation in the channel are calculated from the training set using:

```
mean = tensor_train.mean(dim=(0, 2, 3)),  
std = tensor_train.std(dim=(0, 2, 3))
```

All datasets are normalized using:

$$x' = \frac{x - \mu}{\sigma}$$

- **Step 6: Saving Processed Data.** Preprocessed tensors and label arrays are saved using `torch.save()` into the `./data_preprocessed/` directory. Example files include `tensor_train.pt` and `labels_val.pt`.
- **Step 7: Validation of Preprocessing.** The saved files are reloaded using `torch.load()` to verify correctness, ensuring data integrity and reproducibility.

The comprehensive seven-step data preprocessing pipeline successfully transforms the raw image dataset into a standardized format optimized for deep learning model training

and evaluation. Through systematic implementation of image loading, resizing, normalization, tensor format conversion, and per-channel standardization, the preprocessing procedure ensures consistent data representation across all 10 classification categories. The normalization of pixel values to the $[0, 1]$ range and conversion to float32 format establishes optimal numerical conditions for gradient-based optimization algorithms, while the NCHW tensor format conversion maximizes computational efficiency on GPU architectures. The per-channel standardization using training set statistics ensures that input features maintain zero mean and unit variance, facilitating stable model convergence during training. The creation of label-to-index mappings and persistent storage of preprocessed datasets as PyTorch .pt files enables efficient data loading and reproducible experimental conditions.

B. Model training

1) *Model Architecture and Training Process:* For this study, we employed ResNet34 (Residual Network with 34 layers) as our teacher model architecture due to its proven effectiveness in computer vision tasks and its balance between computational efficiency and representational capacity. ResNet34 belongs to the family of deep residual networks introduced by He et al., which revolutionized deep learning by addressing the vanishing gradient problem through the introduction of skip connections or residual blocks. The architecture consists of 34 layers organized into residual blocks, where each block contains shortcut connections that allow gradients to flow directly to earlier layers during backpropagation. This design enables the training of much deeper networks while maintaining gradient flow and preventing degradation problems commonly observed in very deep networks.

The ResNet34 architecture comprises four main stages of residual blocks with different feature map sizes: the first stage processes 64-channel feature maps, followed by stages with 128, 256, and 512 channels respectively. Each residual block contains two 3×3 convolutional layers with batch normalization and ReLU activation functions. The skip connections in ResNet34 perform identity mapping when input and output dimensions match, or use 1×1 convolutions for dimension matching when spatial dimensions change between stages. This architectural design allows the network to learn residual mappings $F(x) = H(x) - x$, where $H(x)$ represents the desired underlying mapping, making it easier for the network to learn identity functions and enabling effective training of deep networks.

Our training methodology was implemented using the PyTorch framework with CUDA acceleration for GPU computation. The model was configured with a modified final fully connected layer to accommodate the specific number of classes in our dataset (`num_classes`). All layers except the final classification layer were initialized with pretrained ImageNet weights, leveraging transfer learning to improve convergence speed and final performance. To prevent overfitting to the pretrained features, we disabled gradient computation for all pretrained parameters using `param.requires_grad`

= False, allowing only the final classification layer to be trained on our specific dataset.

2) *Training Configuration and Hyperparameters:* The training process utilized carefully selected hyperparameters optimized for the knowledge distillation task. We set the maximum number of training epochs to 10 (`MAX_EPOCH = 10`) based on the principle that pretrained models require fewer epochs to adapt to new tasks compared to training from scratch. Since only the final classification layer requires training while feature extraction layers remain frozen, extensive training would risk overfitting the classifier to the training data without improving feature representations. The limited epoch count also aligns with transfer learning best practices where rapid adaptation is preferred over prolonged training.

The initial learning rate was configured at 0.001 (`INIT_LR = 0.001`) to ensure stable fine-tuning of the pretrained model. This relatively conservative learning rate prevents catastrophic forgetting of previously learned features while allowing sufficient parameter updates for task adaptation. Higher learning rates (e.g., 0.01) would risk destabilizing the pretrained weights, while lower rates (e.g., 0.0001) would result in insufficient learning within the limited epoch budget.

The batch size was set to 64 (`BATCH_SIZE = 64`) to optimize the trade-off between gradient estimation accuracy and computational efficiency, considering our training dataset size of 7,800 samples. This configuration results in approximately 122 batches per epoch ($7800/64 \approx 122$), providing sufficient gradient updates per epoch for stable learning while maintaining reasonable computational overhead. Larger batch sizes (e.g., 128, 256) would reduce the number of gradient updates to 61 or 31 per epoch respectively, potentially insufficient for effective learning within the 10-epoch limit, despite providing more stable gradient estimates. Additionally, larger batches would require significantly more GPU memory and could lead to slower convergence in transfer learning scenarios. Smaller batch sizes (e.g., 16, 32) would increase gradient updates to 488 or 244 per epoch, but would introduce excessive gradient noise and instability, particularly problematic when training only the final layer where stable convergence is crucial.

For optimization, we employed the Adam optimizer (`torch.optim.Adam`) over traditional SGD due to its superior performance in transfer learning scenarios. Adam's adaptive learning rate mechanism is particularly beneficial when training only selected layers, as it automatically adjusts parameter updates based on gradient history. Unlike SGD, which requires manual learning rate scheduling and momentum tuning, Adam provides robust convergence with minimal hyperparameter adjustment.

The loss function utilized was `CrossEntropyLoss` (`nn.CrossEntropyLoss()`), which is the standard loss function for multi-class classification tasks. `CrossEntropyLoss` is widely adopted in computer vision applications due to its mathematical properties that naturally align with classification objectives, combining softmax activation with negative log-likelihood loss to produce probabilistic outputs suitable for measuring prediction confidence and facilitating stable

gradient-based optimization.

3) *Training Process and Validation:* The training loop implemented a comprehensive monitoring system to track model performance and prevent overfitting. For each epoch, the training process involved random permutation of the training data indices to ensure different batch compositions across epochs, which is critical given our data organization structure. Since the dataset is organized with images stored in class-specific subdirectories, each containing 780 images per class, sequential data loading would result in batches containing samples from only one class at a time. This would lead to ineffective learning as the model would encounter homogeneous batches and fail to learn discriminative features across different classes. The random permutation ensures that each batch contains a diverse mix of samples from different classes, enabling the model to learn meaningful inter-class distinctions and preventing gradient updates biased toward individual classes. Following data shuffling, batch-wise processing of input tensors and corresponding labels was performed. During each forward pass, the model processed batches of input images through the ResNet34 architecture, computing classification outputs. The `CrossEntropyLoss` function calculated the discrepancy between predicted and true labels, with gradients computed through backpropagation using `loss.backward()`.

Parameter updates were performed using `optimizer.step()` after zeroing gradients with `optimizer.zero_grad()` to prevent gradient accumulation across batches. Training metrics including loss and accuracy were computed and tracked throughout each epoch. The training accuracy was calculated as the ratio of correctly predicted samples to total samples processed, while the epoch training loss was computed using a weighted averaging approach to ensure accurate representation across batches of different sizes. Specifically, each batch loss was multiplied by its batch size and accumulated (`train_loss += loss.item() * batch_x.size(0)`), then divided by the total number of training samples at the end of each epoch (`epoch_train_loss = train_loss / len(tensor_train)`). This weighted averaging method accounts for potential variations in batch sizes, particularly for the final batch which may contain fewer samples than the standard batch size, ensuring that the reported epoch loss accurately reflects the average loss per sample rather than the average loss per batch.

Validation was conducted after each training epoch using a separate validation dataset. During validation, the model was switched to evaluation mode using `resnet34.eval()`, and gradient computation was disabled using `torch.no_grad()` to improve computational efficiency and prevent parameter updates. Validation metrics mirrored the training metrics, computing validation loss and accuracy to monitor model generalization performance.

4) *Model Checkpointing and Performance Monitoring:* A sophisticated model checkpointing mechanism was implemented to preserve the best-performing model parameters throughout training. The system tracked val-

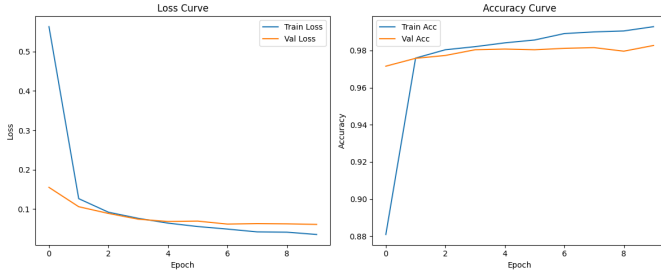


Fig. 1. Training and validation loss and accuracy curves over 10 epochs

validation accuracy across epochs, saving model parameters to 'resnet34_checkpoint.pkl' whenever validation accuracy exceeded the previous best performance (best_val_acc). This approach ensures that the final model represents the configuration with optimal generalization performance rather than simply the final epoch parameters.

Training progress was monitored through comprehensive logging that displayed epoch-wise training and validation metrics. The output format included training loss, training accuracy, validation loss, and validation accuracy for each epoch, along with indicators when improved models were saved. Figure 1 illustrates the training and validation dynamics across 10 epochs. The left subplot shows the loss curves, where both training loss and validation loss consistently decrease over time, indicating effective learning and convergence. The training loss decreases more sharply in early epochs and stabilizes below 0.05, while the validation loss plateaus around 0.07 after epoch 4, suggesting good generalization without significant overfitting.

The right subplot displays the accuracy curves. Training accuracy increases rapidly, surpassing 0.98 by epoch 2 and reaching approximately 0.99 by epoch 9. Validation accuracy also improves steadily and remains close to the training curve, plateauing slightly below 0.98. The consistent gap between training and validation accuracy indicates that the model generalizes well to unseen data without severe overfitting. These trends confirm the effectiveness of the chosen training configuration and the model's stability throughout the training process.

The experimental results demonstrate that fine-tuning only the final classification layer of a pretrained ResNet34 model is sufficient to achieve high classification accuracy on the target dataset. Despite freezing all other layers, the model exhibits rapid convergence and strong generalization performance. This confirms that the features learned from large-scale datasets like ImageNet can be effectively transferred to downstream tasks with minimal additional training, validating the efficiency of the transfer learning strategy adopted in this study.

C. Distillation process

The knowledge distillation framework implemented in this study follows a teacher-student paradigm where a larger, more complex teacher model transfers its learned representations to a smaller, more efficient student model. This approach

addresses the fundamental trade-off between model performance and computational efficiency, particularly crucial for deployment in resource-constrained environments.

1) *Teacher-Student Architecture Comparison:* The teacher model employs ResNet-34, a deeper residual network architecture containing 34 layers with approximately 21.8 million parameters. ResNet-34 incorporates four residual blocks with varying numbers of layers: the first block contains 3 layers, the second contains 4 layers, the third contains 6 layers, and the fourth contains 3 layers.

In contrast, the student model utilizes ResNet-18, a shallower architecture with 18 layers containing approximately 11.7 million parameters. ResNet-18 follows a similar structural design but with reduced depth: each residual block contains only 2 layers instead of the varying depths found in ResNet-34. This reduction in depth results in approximately 46% fewer parameters while maintaining the essential residual learning framework. The architectural simplification enables faster inference times and reduced memory consumption, making it suitable for deployment scenarios where computational resources are limited.

2) *Choice of hyperparameters:* The training configuration incorporates carefully selected hyperparameters optimized for knowledge distillation effectiveness. The initial learning rate is set to 1×10^{-3} , chosen through empirical validation to provide stable convergence while maintaining reasonable training time. A StepLR scheduler with step_size=10 and gamma=0.1 implements learning rate decay every 10 epochs, reducing the global learning rate from 0.001 to 0.0001 after 10 epochs, then to 0.00001 after 20 epochs, and so forth.

This global scheduling works synergistically with Adam's parameter-specific adaptivity: while Adam dynamically adjusts individual parameter learning rates based on gradient statistics, the scheduler ensures the overall learning magnitude decreases progressively throughout training. This dual-level approach prevents late-stage oscillations around optimal solutions.

The batch size of 64 and maximum epoch limit of 40 provide sufficient training iterations.

3) *Knowledge Distillation Implementation:* The knowledge distillation process operates through a simultaneous training mechanism where both teacher and student models process identical input data during each training iteration. The pre-trained teacher model (ResNet-34) generates soft probability distributions that encapsulate its learned feature representations, while the student model (ResNet-18) attempts to mimic these distributions alongside learning from ground truth labels.

The distillation process builds on this idea by combining two types of supervision signals: hard labels and soft labels. Hard labels are the conventional one-hot encoded vectors—like [1, 0, 0] for a cat—which only tell the model which class is correct, without giving any information about the relationships between classes. In contrast, soft labels, such as [0.7, 0.25, 0.05] reveal how the teacher model views other classes in relation to the correct one, offering richer learning signals.

To train the student, we use a hybrid loss function:

$$L_{\text{distillation}} = \alpha \times T^2 \times L_{\text{soft}} + (1 - \alpha) \times L_{\text{hard}}$$

where L_{soft} is the Kullback-Leibler divergence between the teacher’s and student’s softened predictions, and L_{hard} is the regular cross-entropy loss with the ground truth. The T^2 term adjusts for the smaller gradient magnitudes caused by temperature scaling, ensuring that soft targets still meaningfully contribute to the training.

The temperature parameter T introduces a crucial softening mechanism to the output probability distribution. When $T = 1$, the teacher model typically produces sharp predictions (e.g., [0.95, 0.04, 0.01] for a cat image), offering limited insight into how the model differentiates between similar classes. By increasing the temperature to $T = 5$, the same logits yield a softened distribution such as [0.6, 0.3, 0.1], indicating that the teacher perceives a closer resemblance between cats and dogs than between cats and birds. This softened output captures meaningful class-level similarities and semantic structure that would otherwise be lost, thereby providing richer supervision for the student model.

We set $\alpha = 0.5$ to give equal weight to both types of signals, so the student learns from both the teacher’s ”intuition” and the actual correct labels. During training, the teacher model remains fixed (in evaluation mode), providing a stable source of guidance while the student gradually learns to mimic its behavior more efficiently.

4) *Training Process and Optimization:* At the beginning of each epoch, the training data is randomly permuted to avoid bias from fixed data ordering. For each batch, both the teacher and student networks perform forward propagation on the same input images, producing their respective output logits. The distillation loss is computed by combining the Kullback-Leibler divergence between the softened student and teacher outputs with the standard cross-entropy loss relative to the ground truth. Importantly, the backpropagation process only updates the student model’s parameters, while the teacher remains fixed in evaluation mode. This targeted update strategy ensures that the student benefits from the teacher’s guidance without altering the pretrained teacher weights.

Validation is conducted at the end of each epoch using a separate validation dataset. During this phase, the student model is switched to evaluation mode to disable dropout and batch normalization updates. Both teacher and student models generate predictions on the validation set, and the same hybrid distillation loss function is used to evaluate student performance. Validation accuracy is computed based on the student’s predictions alone.

IV. RESULT

To evaluate the performance of the proposed knowledge distillation (KD) approach in this task, we compared the teacher model, the distilled student model, and a baseline non-distilled ResNet18. The evaluation covered accuracy, loss, precision, recall, F1 score, top-3 accuracy, and computational efficiency.

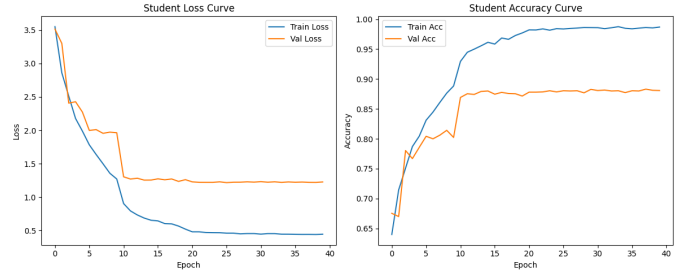


Fig. 2. KD Training and validation loss and accuracy over 40 epochs

A. Loss and Accuracy Comparison

As shown in Figure 1 and Figure 2, the training and validation loss and accuracy curves demonstrate the convergence behaviors of the teacher and student models.

- Teacher Train Accuracy: 0.9927
- Teacher Validation Accuracy: 0.9827
- Student Train Accuracy: 0.9876
- Student Validation Accuracy: 0.8831

While the teacher model achieved higher validation accuracy and converged within fewer epochs, the student model also demonstrated effective convergence, with slightly lower but acceptable accuracy given its lighter architecture. The teacher model’s low loss and high accuracy across both training and validation sets suggest strong generalization capability. In contrast, the student model exhibited slightly higher validation loss overall but maintained stable accuracy across 40 epochs, indicating successful knowledge transfer.

B. Other Model Metrics

- KD Precision: 0.8827
- KD Recall: 0.8831
- KD F1 Score: 0.8826
- KD Top-3 Accuracy: 0.9669

In Figure 3, the student model demonstrated consistent classification performance across all evaluation metrics. A precision of 88.27% means that most predicted classes are correct, while a recall of 88.31% indicates that the model successfully identified the majority of true instances across all classes. The F1 score of 88.26% confirms the model maintains a good balance between precision and recall. The top-3 accuracy of 96.69% suggests that even when the top-1 prediction is incorrect, the correct label often appears among the top-3 predicted classes.

C. Comparison with Non-distilled ResNet18

- ResNet18 (No KD) Train Accuracy: 0.9978
- ResNet18 (No KD) Val Accuracy: 0.8369

In Figure 4 Compared with the distilled model, the non-KD ResNet18 exhibited high training accuracy but suffered from lower validation accuracy and unstable convergence, as seen in Fig. 5. Despite training for 60 epochs, the model showed significant variance in validation accuracy, implying overfitting and lack of robustness. In contrast, the student model trained

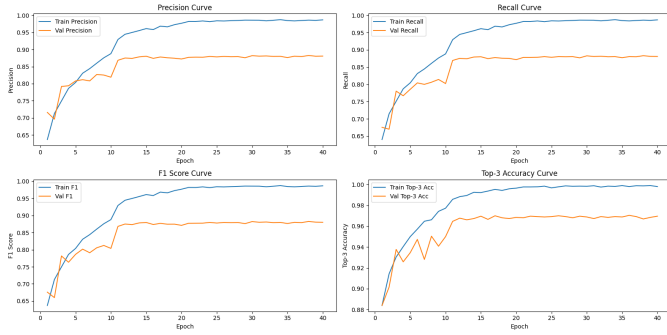


Fig. 3. Other Metrics for student KD

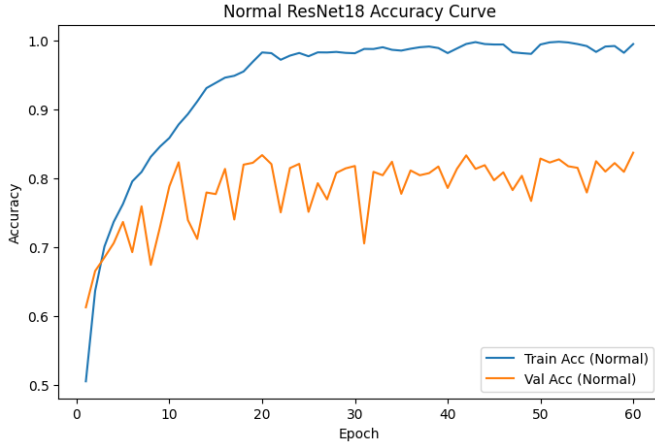


Fig. 4. Resnet18 without KD Accuracy

via KD achieved better generalization in fewer epochs and with more stable performance.

V. DISCUSSION

The experimental results demonstrate that the knowledge distillation approach successfully transferred the predictive capability of the deeper ResNet34 teacher model to a lightweight ResNet18 student model. The student model achieved strong performance with significantly fewer parameters, validating the effectiveness of the distillation method in enhancing efficiency without compromising classification accuracy.

All evaluation metrics support the conclusion that the distilled student model performs competitively. As shown in Section IV, the student model achieved 88.27% precision, meaning that when it predicted a certain class, it was correct over 88% of the time. The recall of 88.31% indicates its ability to detect a high proportion of all relevant class instances. The F1 score of 88.26% reflects a strong balance between precision and recall, while a top-3 accuracy of 96.69% confirms the model's robustness in ranking correct predictions within its top three choices. These metrics collectively confirm the effectiveness of the student model in a 10-class classification task and its ability to generalize well across all categories.

Compared to the non-distilled ResNet18 model, which exhibited larger accuracy fluctuations and signs of overfitting

despite longer training, the distilled student model converged faster and achieved better generalization, as evidenced by smoother validation accuracy and lower loss. Moreover, it reached comparable accuracy to the teacher model with fewer epochs and lighter architecture, demonstrating superior computational efficiency.

Nevertheless, there are limitations. The current framework only distills logits and does not explore intermediate feature representations or more advanced distillation techniques. Potential improvements include integrating attention transfer or contrastive distillation to further enhance representation learning. Additionally, applying this framework to larger datasets or real-world tasks would validate its scalability and robustness.

From a learning perspective, implementing knowledge distillation provided valuable insight into model compression techniques. It illustrated how simple strategies, such as temperature-scaled soft targets and hybrid loss functions, can significantly improve the performance of compact models. This understanding reinforces the practicality of knowledge distillation as a tool for efficient model deployment in real-world scenarios.

VI. CONCLUSION

This study demonstrates that knowledge distillation can effectively compress a large-scale teacher model into a compact student model without sacrificing accuracy. By distilling a pretrained ResNet34 into a lightweight ResNet18, the proposed framework achieved competitive classification performance on a 10-class image dataset while significantly reducing model complexity and computational cost. The student model maintained high precision, recall, and F1 scores, indicating its strong generalization and reliability across categories.

The findings validate the practicality of knowledge distillation for deploying efficient deep learning models in resource-limited environments. Future research can explore alternative distillation techniques, such as attention transfer or intermediate feature alignment, to further improve performance. Additionally, applying the approach to more complex datasets or real-world applications would provide stronger evidence of scalability and robustness.

Overall, this work contributes a simplified yet effective knowledge distillation framework that balances accuracy and efficiency. It highlights the value of distillation as a tool for building deployable deep learning models while enhancing understanding of student-teacher training dynamics.

REFERENCES

- [1] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531, 2015.
- [2] J. Ba and R. Caruana, "Do deep nets really need to be deep?" in Advances in Neural Information Processing Systems, vol. 27. Curran Associates, Inc., 2014.
- [3] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for thin deep nets," arXiv preprint arXiv:1412.6550, 2014.
- [4] S. Zagoruyko and N. Komodakis, "Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer," arXiv preprint arXiv:1612.03928, 2016.

- [5] T. Furlanello, Z. C. Lipton, M. Tschannen, L. Itti, and A. Anandkumar, "Born again neural networks," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1607–1616.
- [6] Y. Zhang, T. Xiang, T. M. Hospedales, and H. Lu, "Deep mutual learning," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4320–4328.